DATA SCIENCE & ANALYTICS

MODULE 02 SESSION 10

Window functions, and beyond the sandbox

How do I compare a row with its group without losing the rows?

Tarek Atwan · Lead Instructor

Mon 12 Oct · 4:00–8:00 PM

BY THE END OF TODAY, YOU CAN

Class objectives

01 Compare each row with its group

02 Rank inside a group and keep the top N

03 Use a window for trends, gaps and duplicates

04 Query a file in place; read less with partitions

05 Load the Assignment 1 data

EVERYONE DO

Step 1.1

Warm-up

Four questions on D09, and one that sets up today.

Suggested time · 10 minutes

PART 02

OVER keeps the rows

GROUP BY answers a question about a group by collapsing it. A window keeps every row.

INSTRUCTOR DO

Step 2.1

A window keeps the rows

“Which months were better than our average month?” A third way.

Suggested time · 12 minutes

labs/D10/01-Ins-Window-Keeps-Rows/README.md

QUERY 1 · GROUP BY COLLAPSES

The average month: one row

AVG collapses the 24 months into one number.

```
WITH monthly AS (
  SELECT DATE_TRUNC(order_date, MONTH) AS month,
         SUM(order_total) AS revenue
  FROM nakheel.orders
  WHERE status = 'completed'
  GROUP BY month
)
SELECT ROUND(AVG(revenue), 2) AS avg_month
FROM monthly;
```

→ 1 row: 13,167.65

QUERY 2 · OVER () KEEPS THEM

The average beside every month

OVER (): the window is every row.

```
WITH monthly AS (
  SELECT DATE_TRUNC(order_date, MONTH) AS month,
         SUM(order_total) AS revenue
  FROM nakheel.orders
  WHERE status = 'completed'
  GROUP BY month
)
SELECT month,
       revenue,
       ROUND(AVG(revenue) OVER (), 2) AS avg_month
FROM monthly
ORDER BY month;
```

→ 24 rows · 13,167.65 on every row

DEFINITION

A window function calculates over a set of rows, the window, and writes the answer on each row instead of collapsing them.

GROUP BY OR OVER?

Collapse the rows, or keep them

GROUP BY: one row

avg_month
13,167.65

OVER (): every row, the answer beside it

month	revenue	avg_month
2024-09	11,268.00	**13,167.65**
2024-10	11,582.00	**13,167.65**
2024-11	10,054.50	**13,167.65**
2024-12	12,947.00	**13,167.65**
2025-01	13,096.50	**13,167.65**
…	…	…

The average is the same; the rows are kept. 24 rows in all.

PLANNED DEAD END

A window in WHERE

WHERE runs before the window is calculated.

```
WITH monthly AS ( … )
SELECT month, revenue
FROM monthly
WHERE revenue > AVG(revenue) OVER ();
```

```
BigQuery refuses: a window function cannot go in WHERE
```

Put the window in a CTE, then filter in the next step.

QUERY 3 · THE WINDOW IN A CTE

Then filter in the next step

The same 8 months as D08 and D09.

```
WITH monthly AS (
  SELECT DATE_TRUNC(order_date, MONTH) AS month,
         SUM(order_total) AS revenue
  FROM nakheel.orders
  WHERE status = 'completed'
  GROUP BY month
),
with_average AS (
  SELECT month, revenue,
         AVG(revenue) OVER () AS avg_month
  FROM monthly
)
SELECT month, ROUND(revenue, 2) AS revenue
FROM with_average
WHERE revenue > avg_month
ORDER BY month;
```

→ 8 rows · April 2026, 20,237.00

QUERY 4 · A PREVIEW

BIGQUERY-SPECIFIC

The shortcut: QUALIFY

QUALIFY filters on the window, so no second CTE.

```
WITH monthly AS (
  SELECT DATE_TRUNC(order_date, MONTH) AS month,
         SUM(order_total) AS revenue
  FROM nakheel.orders
  WHERE status = 'completed'
  GROUP BY month
)
SELECT month,
       ROUND(revenue, 2) AS revenue
FROM monthly
QUALIFY revenue > AVG(revenue) OVER ()
ORDER BY month;
```

→ the same 8 rows · no second CTE

EVERYONE DO

Step 2.2

Each product’s share of its category

“What share of its category’s revenue does each product bring?”

Suggested time · 15 minutes

labs/D10/02-Evr-Share-Of-Category/README.md

STEP 1 · ONE ROW PER PRODUCT

Completed revenue per product

The D06 three-table join, grouped by category and product.

```
WITH product_revenue AS (
  SELECT p.category,
         p.product_name,
         SUM(i.qty * i.price) AS revenue
  FROM nakheel.order_items AS i
  INNER JOIN nakheel.orders AS o
    ON i.order_id = o.order_id
  INNER JOIN nakheel.products AS p
    ON i.product_id = p.product_id
  WHERE o.status = 'completed'
  GROUP BY p.category, p.product_name
)
SELECT *
FROM product_revenue;
```

→ 56 rows

PARTITION BY CATEGORY

One window per category

SUM(revenue) OVER (PARTITION BY category): the band’s total on each of its rows.

category	product_name	revenue	category_revenue
Outerwear	Denim jacket	12,818.00	**76,637.00**
Outerwear	Wool overcoat	10,320.00	**76,637.00**
Shoes	Ankle boot	12,000.00	**62,373.50**
Shoes	Running shoe	11,594.00	**62,373.50**
Tops	White thobe	9,768.00	**40,588.00**
Tops	Linen shirt	7,614.00	**40,588.00**

Two products per category shown; the query keeps all 56 rows.

Each product against its own category

PARTITION BY category cuts the rows into one window per category.

```
SELECT category,
       product_name,
       revenue,
       SUM(revenue) OVER (PARTITION BY category)
         AS category_revenue,
       ROUND(100 * revenue
         / SUM(revenue) OVER (PARTITION BY category), 1)
         AS share_pct
FROM product_revenue
ORDER BY share_pct DESC;
```

→ 56 rows · White thobe 24.1% of Tops

STUDENT DO

Activity: Window practice

In this activity, you will compare orders with their channel, customers with their segment, and months with their year.

Suggested time

12 minutes

labs/D10/03-Stu-Window-Practice/README.md

STUDENT DO · 12 MINUTES

Activity: Window practice

INSTRUCTIONS	1. Each completed order next to its channel’s average. 2. Customers above their own segment’s average, counted per segment. 3. Each month’s share of its year’s revenue.
HINT	A window cannot go in WHERE: put it in a CTE, then filter.
BONUS	Which month of 2025 took the biggest share?

Full steps and Check yourself: labs/D10/03-Stu-Window-Practice/README.md

REVIEW

Time's up. Let's review.

Window practice

Solved: labs/D10/03-Stu-Window-Practice/Solved/window_practice.sql

QUESTION 3 · CHECK WHAT IS IN THE WINDOW

28.2%

December 2024’s share of 2024. But 2024 has only four months in the data: a share is fair only when the windows are alike.

labs/D10/03-Stu-Window-Practice · 12,947.00 of 45,851.50

PART 03

Rank inside a group

Number the rows in order, and decide what a tie gets.

INSTRUCTOR DO

Step 3.1

Rank within a group, and what a tie does

“Rank the Sportswear products by units sold.”

Suggested time · 12 minutes

labs/D10/04-Ins-Rank-Ties/README.md

ROW_NUMBER, RANK and DENSE_RANK

The same order; they differ only on ties.

```
SELECT product_name,
       units,
       ROW_NUMBER() OVER (ORDER BY units DESC, product_name) AS row_num,
       RANK()       OVER (ORDER BY units DESC) AS rank_num,
       DENSE_RANK() OVER (ORDER BY units DESC) AS dense_num
FROM product_units
WHERE category = 'Sportswear'
ORDER BY units DESC, product_name;
```

→ 9 rows

SPORTSWEAR · TWO PRODUCTS ON 284

What a tie gets

product_name	units	ROW_NUMBER	RANK	DENSE_RANK
Yoga mat	423	5	5	5
Compression top	284	6	**6**	**6**
Gym shorts	284	7	**6**	**6**
Football jersey	194	8	**8**	**7**
Training jacket	141	9	9	8

Without a second sort column, ROW_NUMBER can swap the two 284s between runs.

WHICH ONE TO USE

Choose by what a tie should get

Function	A tie gets	Use it when
ROW_NUMBER	different numbers	you need exactly N rows
RANK	the same, then a gap	a league table: joint 6th, then 8th
DENSE_RANK	the same, no gap	“the top three places”

EVERYONE DO

Step 3.2

The top three in every category

“What are the three best-selling products by revenue in each category?”

Suggested time · 13 minutes

labs/D10/05-Evr-Top-Three/README.md

STEP 3 · QUALIFY

Filter on the window

QUALIFY filters a window's result, as HAVING filters an aggregate.

```
SELECT category,
       product_name,
       revenue,
       RANK() OVER (PARTITION BY category
                    ORDER BY revenue DESC)
         AS revenue_rank
FROM product_revenue
QUALIFY revenue_rank <= 3
ORDER BY category, revenue_rank;
```

→ 18 rows · three per category

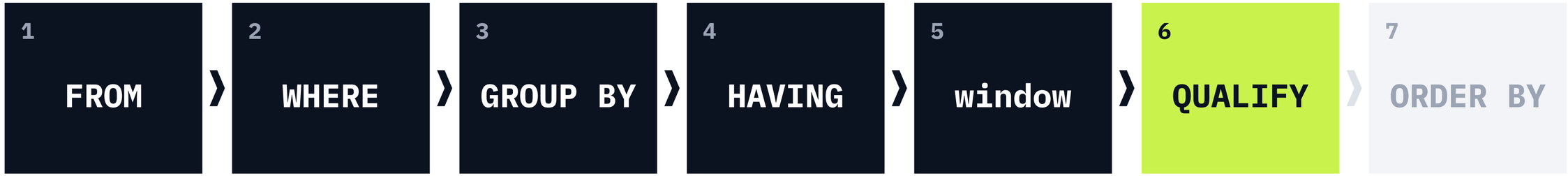
BIGQUERY-SPECIFIC

The order a query runs in

YOU WRITE

SELECT FROM WHERE GROUP BY HAVING QUALIFY ORDER BY

BIGQUERY RUNS



The window is calculated after HAVING. QUALIFY runs after the window, so it can use it.

THREE FILTERS

WHERE, HAVING or QUALIFY?

	Filters	Runs after	Can test	Example
WHERE	rows	FROM	a row’s own columns	status = 'completed'
HAVING	groups	GROUP BY	aggregates: SUM, COUNT	SUM(order_total) > 50000
QUALIFY	rows	the window	window results: RANK, AVG() OVER	revenue_rank <= 3

What are you testing? A row’s value: WHERE. A group’s total: HAVING. A window’s answer: QUALIFY.

PART 04

Trends and four more jobs

A running total, the month before, four more jobs for a window, and one query that uses everything since D03.

INSTRUCTOR DO

Step 4.1

Running totals and the month before

“What has the business made so far, month by month?”

Suggested time · 12 minutes

labs/D10/06-Ins-Running-Total-Lag/README.md

A RUNNING TOTAL

The window grows one month at a time

ORDER BY inside OVER: from the first row up to this one.

```
SELECT month,
       revenue,
       SUM(revenue) OVER (ORDER BY month)
         AS running_total
FROM monthly
ORDER BY month;
```

→ 24 rows · last row 316,023.50

The last row is a number you know

month	revenue	running_total
2024-09	11,268.00	11,268.00
2024-10	11,582.00	22,850.00
2024-11	10,054.50	32,904.50
…	…	…
2026-07	16,938.50	302,962.00
2026-08	13,061.50	**316,023.50**

316,023.50: completed revenue, checked since D04.

The change on the previous month

LAG(revenue) reads revenue from the row before, in the window's order.

```
SELECT month,
       revenue,
       LAG(revenue) OVER (ORDER BY month)
         AS previous_month,
       revenue - LAG(revenue) OVER (ORDER BY month)
         AS change
FROM monthly
ORDER BY month;
```

→ April 2026 +5,792.50 · May 2026 −7,473.00

INSTRUCTOR DO

Step 4.2

Four more jobs for a window

“What else is a window for?” Three families, and four jobs beyond ranking.

Suggested time · 10 minutes

labs/D10/13-Ins-More-Window-Jobs/README.md

THREE FAMILIES

Every window function is one of three kinds

Family	Functions	Each row gets
Aggregate	SUM, AVG, COUNT, MIN, MAX with OVER	a total or average of its window
Ranking	ROW_NUMBER, RANK, DENSE_RANK, NTILE, PERCENT_RANK	its position in the window
Value	LAG, LEAD, FIRST_VALUE, LAST_VALUE	a value from another row

Today: SUM, AVG, ROW_NUMBER, RANK, DENSE_RANK and LAG.

What a window does at work

Job	Function	Where
A share of a total	SUM() OVER (PARTITION BY)	2.2
The top N in each group	RANK() + QUALIFY	3.2
Running total, year to date	SUM() OVER (ORDER BY)	4.1
Change on the period before	LAG()	4.1
One row per key	ROW_NUMBER() … = 1	**now**
Time between events	LAG() + DATE_DIFF	**now**
Equal groups	NTILE(4)	**now**
Moving average	AVG() OVER (ROWS …)	**now**

JOB 1 · ONE ROW PER KEY

Remove a duplicate person

Omar Nasser, Irbid: C-1204 (3 orders) and C-1401 (18). Keep the busier account.

```
WITH customer_orders AS ( … )
SELECT customer_id, customer_name,
       city, completed_orders
FROM customer_orders
QUALIFY ROW_NUMBER() OVER (
          PARTITION BY customer_name, city
          ORDER BY completed_orders DESC,
                   customer_id) = 1;
```

→ 499 rows from 500 · C-1401 kept

JOB 2 · TIME BETWEEN EVENTS

Days since the same customer’s last order

DATE_DIFF counts the days; in BigQuery the later date comes first.

```
WITH with_previous AS (
  SELECT customer_id, order_id, order_date,
         LAG(order_date) OVER (
           PARTITION BY customer_id
           ORDER BY order_date, order_id)
           AS previous_order_date
  FROM nakheel.orders
  WHERE status = 'completed'
)
SELECT customer_id, order_date,
       DATE_DIFF(order_date, previous_order_date, DAY)
         AS days_since_previous
FROM with_previous;
```

→ 2,771 rows · 2,319 gaps · average 60.8 days

Customers in four equal groups

NTILE(4) cuts the rows, in order, into four groups of equal size.

```
WITH customer_revenue AS ( … ),
quartiles AS (
  SELECT customer_id, revenue,
         NTILE(4) OVER (ORDER BY revenue DESC,
                        customer_id) AS quartile,
         100 * revenue / SUM(revenue) OVER ()
           AS share_pct
  FROM customer_revenue
)
SELECT quartile, COUNT(*) AS customers,
       ROUND(SUM(share_pct), 1) AS share_pct
FROM quartiles
GROUP BY quartile;
```

→ 4 rows of 113 · top quarter 80.6% · bottom 1.8%

JOB 4 · A MOVING AVERAGE

Smooth the monthly line

ROWS BETWEEN sets a frame: this month and the two before it.

```
WITH monthly AS ( … )
SELECT month, revenue,
       AVG(revenue) OVER (ORDER BY month
          ROWS BETWEEN 2 PRECEDING
          AND CURRENT ROW) AS moving_avg_3m
FROM monthly;
```

→ April 2026: 20,237.00 alone, 16,016.00 smoothed

STUDENT DO

Activity: Rank and trend

In this activity, you will find each channel’s biggest orders, each channel’s monthly change, and our three best months.

Suggested time

18 minutes

`labs/D10/07-Stu-Rank-And-Trend/README.md`

STUDENT DO · 18 MINUTES

Activity: Rank and trend

INSTRUCTIONS	1. The three biggest completed orders in each channel. 2. Each channel’s monthly change, and the biggest jump. 3. Our three best months, with their rank.
HINT	LAG needs PARTITION BY channel, or the channels run into each other.
BONUS	A running total for each channel.

Full steps and Check yourself: labs/D10/07-Stu-Rank-And-Trend/README.md

REVIEW

Time's up.
Let's review.

Rank and trend

`Solved: labs/D10/07-Stu-Rank-And-Trend/Solved/rank_and_trend.sql`

EVERYONE DO

Step 4.5

One query that uses the whole module

“For each segment, our three biggest customers, their city, and their share of the segment.”

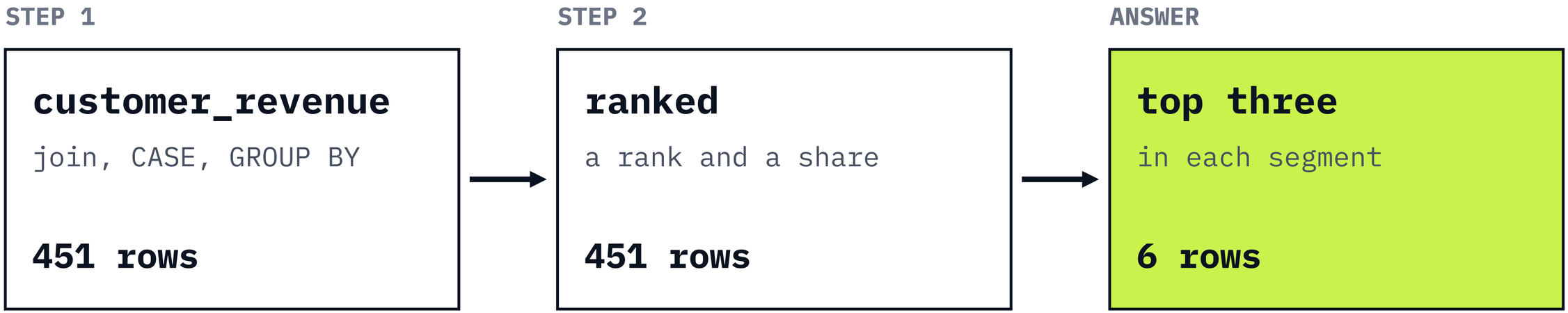
Suggested time · 20 minutes

labs/D10/08-Evr-Whole-Module/README.md

PLAN THE STEPS FIRST

Two named steps, then the answer

Step 1 uses D04–D07. Step 2 uses today. Check the rows after each.



STEP 2 · TWO WINDOWS

A rank and a share, inside each segment

Then WHERE segment_rank <= 3 in the final SELECT.

```
ranked AS (
  SELECT customer_id, customer_name,
         city_clean, segment_clean, revenue,
         RANK() OVER (PARTITION BY segment_clean
                      ORDER BY revenue DESC)
           AS segment_rank,
         ROUND(100 * revenue / SUM(revenue)
           OVER (PARTITION BY segment_clean), 1)
           AS share_of_segment
  FROM customer_revenue
)
```

REVIEW

Time's up. Let's review.

The whole module

Solved: labs/D10/08-Evr-Whole-Module/Solved/whole_module.sql

THE ANSWER · 6 ROWS

One customer: 22.9% of wholesale

segment	rank	customer	city	revenue	share
wholesale	1	Leen Najjar	amman	35,338.50	**22.9**
wholesale	2	Omar Masri	aqaba	18,676.00	12.1
wholesale	3	Salma Odeh	irbid	13,445.00	8.7
retail	1	Tala Rawashdeh	irbid	8,571.50	5.3
retail	2	Jana Issa	amman	8,196.00	5.1
retail	3	Lina Darwish	karak	6,324.50	3.9

Break

Back at 7:00 PM

After the break, part two: where a warehouse’s data lives and how it reads less, then Assignment 1. Bring the football files.

PART 05

Beyond the sandbox

Where a cloud warehouse’s data lives, and how it reads less.

INSTRUCTOR DO

Step 5.1

Where a warehouse’s data lives

Storage and compute; tables that hold rows, and tables that only point at files.

Suggested time · 12 minutes

STORAGE AND COMPUTE

Two separate parts of a cloud warehouse

Storage

tables and files; cheap; always there

Compute

starts when you run a query; billed by bytes read

DEFINITION

An external table stores only a file’s address, format and columns. The rows stay in the file, and every query reads them where they are.

NATIVE OR EXTERNAL?

Load the rows, or read them in place

	Native table	External table
Rows are	in BigQuery’s storage	in a file in Cloud Storage
Storage size	shown in Details	none
File changes	upload again	the next query reads it
Speed	fast	slower: read every time

THE SAME IDEA, TWO WAREHOUSES

BigQuery over Cloud Storage, Snowflake over S3

BigQuery

```
CREATE EXTERNAL TABLE files.us_states (…)
OPTIONS (format = 'CSV',
  uris = ['gs://bucket/path/*.csv']);
```

Snowflake

```
CREATE STAGE lake URL = 's3://bucket/path/';
CREATE EXTERNAL TABLE us_states
  LOCATION = @lake
  FILE_FORMAT = (TYPE = CSV);
```

Databricks and Microsoft Fabric read lake files the same way.

IN THE SANDBOX

Why we read a public file

01 Your own Cloud Storage bucket needs billing

02 The sandbox has no billing account

03 Google publishes sample files anyone can read

04 So we point an external table at one

EVERYONE DO

Step 5.2

Query a file where it lives

An external table on Google’s public sample file of US states.

Suggested time · 11 minutes

labs/D10/09-Evr-External-Table/README.md

STEP 2 · THE EXTERNAL TABLE

Store the definition, not the rows

gs:// is Cloud Storage; cloud-samples-data is Google's public bucket.

```
CREATE OR REPLACE EXTERNAL TABLE files.us_states (
  name STRING,
  post_abbr STRING
)
OPTIONS (
  format = 'CSV',
  uris = ['gs://cloud-samples-data/bigquery/us-states/us-states.csv'],
  skip_leading_rows = 1
);
```

STEP 4 · QUERY IT LIKE ANY TABLE

Which states start with W?

The rows are read from the file as the query runs.

```
SELECT name, post_abbr
FROM files.us_states
WHERE name LIKE 'W%'
ORDER BY name;
```

```
→ 4 rows · Washington … Wyoming
```

STEP 5 · WHICH FILE?

Every row knows its file

_FILE_NAME is a hidden column of Cloud Storage external tables.

```
SELECT _FILE_NAME AS source_file,
       COUNT(*) AS states
FROM files.us_states
GROUP BY source_file;
```

→ 1 row · the gs:// address · 50

INSTRUCTOR DO

Step 5.3

Read less: partitions and clusters

“The bytes estimate is the bill. How does a warehouse read less?”

Suggested time · 10 minutes

labs/D10/14-Ins-Partitions/README.md

DEFINITION

A partition is one part of a table, split by the values of one column. A filter on that column reads only the parts it needs.

PARTITION PRUNING

A query for April reads one month

Picture nakheel.orders split by month: 24 partitions.

2024-09	2024-10	2024-11	2024-12	2025-01	2025-02	2025-03	2025-04	2025-05	2025-06	2025-07	2025-08
2025-09	2025-10	2025-11	2025-12	2026-01	2026-02	2026-03	**2026-04**	2026-05	2026-06	2026-07	2026-08

```
WHERE order_date >= '2026-04-01' AND order_date < '2026-05-01'
```

BigQuery reads one partition: 144 of the 3,000 orders.

THREE KINDS OF PARTITION

Split by a date, a load day or a number

Type	Split by	Example
Time-unit column	a DATE or TIMESTAMP column; day, month or year	order_date, by month
Ingestion time	the day the rows were loaded: _PARTITIONTIME	daily loads
Integer range	a whole number: start, end, interval	customer numbers by 100

Source: Google Cloud, Introduction to partitioned tables, read 10 October 2026.

Partition and cluster when you create the table

PARTITION BY here decides how the table is stored.

```
CREATE TABLE nakheel.orders_partitioned
PARTITION BY DATE_TRUNC(order_date, MONTH)
CLUSTER BY channel
AS
SELECT *
FROM nakheel.orders;
```

Do not run this one in the sandbox: the catch is two slides on.

PARTITION OR CLUSTER?

Skip whole parts, or skip blocks inside them

	Partition	Cluster
Splits or sorts by	one column	up to four columns
The saving shows	in the estimate, before the run	in bytes billed, after the run
Worth it	from about 10 GB a partition	on smaller tables too

Nakheel is about half a megabyte. At work, nobody would partition it.

LIVE · TYPE, DO NOT RUN

Read two estimates on a real table

refresh_date is the partition column of Google's top_terms table.

```
SELECT term, rank
FROM `bigquery-public-data.google_trends.top_terms`
WHERE refresh_date =
      DATE_SUB(CURRENT_DATE(), INTERVAL 1 DAY);
```

→ with the filter: a fraction of the bytes

THE SANDBOX CATCH

60 days

In the sandbox, partitions expire 60 days after their date. Nakheel ends on 31 August 2026, so a copy partitioned by order_date would lose almost every partition at once.

Google Cloud, BigQuery sandbox, read 10 October 2026

NEEDS A BILLING ACCOUNT

Not in the free sandbox

Feature	What it does
Cloud Storage buckets	your own files for loads and external tables
Data lineage	which tables each table was built from
Profiling and quality scans	column profiles; rules such as “no NULL IDs”
Scheduled queries	run a query every morning
INSERT, UPDATE, DELETE	change rows in place
BigQuery Graph (GQL)	graph queries; an Enterprise reservation

Works in the sandbox: CREATE TABLE, CREATE VIEW, CREATE TABLE AS SELECT. Who sees a dataset: roles on the project, dataset or table.

Sources: Google Cloud documentation, checked 9 and 10 October 2026.

PART 06

Assignment 1

Jordan on the world stage, in SQL. Load the data before you leave.

INSTRUCTOR DO

Step 6.1

Assignment 1 briefing

The scenario, the data, the twelve questions and how they are graded.

Suggested time · 10 minutes

assignments/A01-SQL-Football/README.md

THE SCENARIO

1872

to 2026: every men’s international match, including Jordan’s first World Cup in June 2026 and the Asian Cup final.

International football results, CC0 · 49,807 matches · course copy 9 October 2026

THE DATA

Three tables, joined on match_id

matches
match_id
match_date
home_team
away_team
home_score
away_score
tournament

goals
goal_id
match_id
team
scorer
minute
own_goal
penalty

shootouts
match_id
winner
first_shooter

Primary key Foreign key

49,807 matches · 47,914 goals · 694 shootouts

FOUR PARTS, TWELVE QUESTIONS

Everything from D03 to D10

Part	Questions	Uses
1 · Load and check	Q1–Q2	subquery in SELECT, UNION ALL, CASE, a view
2 · Jordan in numbers	Q3–Q6	GROUP BY, COUNTIF, HAVING, ORDER BY
3 · Goals and coverage	Q7–Q9	running total, JOIN, LEFT JOIN audit
4 · World Cup 2026	Q10–Q12	CTE and subquery, DENSE_RANK, LAG

HOW IT IS GRADED

One .sql file: query, row count, one sentence

Criterion	Weight
Correct results	40%
Row counts	20%
The answer in words	20%
Readable SQL	20%

DATES

Issued today.
Due before 4:00 PM, Wednesday 21 October.
Help: Tuesday 20 October, 4:00–6:00 PM.

EVERYONE DO

Step 6.2

Load the Assignment 1 data

Create football, upload three files, check three counts.

Suggested time · 12 minutes

labs/D10/11-Evr-Upload-Football/README.md

LIVE DEMO

Load the football data

01 Download the three CSVs

02 Create dataset football, Multi-region US

03 Create table → Upload → Auto detect

04 Header rows to skip: 1

Then count: 49,807 · 47,914 · 694

CLOSE

What you can do now

- Compare each row with its group
- Rank inside a group; keep the top N
- Running totals, LAG, more jobs
- Query a file; read less

NEXT · D11 Machine learning, from bi.customer_summary. Finish the D08 lab’s C1.